

逐笔成交量化作业

代码讲解与结果复盘

R Markdown 风格 | 题目要求 · 关键代码 · 输出表格 · 结果解释 · 验证结论

数据	因子	回测	LSTM
302 日 x 300 股 11 个字段	4 个 IC + 五分组	4 个版本 Top 10	PyTorch 流程完成

验证状态：PASS | 单元测试：4/4 通过

数据区间：2025-04-01 至 2026-06-30
生成日期：2026-07-28 | 结果以当前工程产物为准

目录

本报告按代码执行链路组织；页码由最终 PDF 自动生成。

报告说明	2
一、题目要求与完成状态	2
二、项目结构和代码分工	3
三、数据降采样与复权	3
四、因子构建	5
五、因子清洗、IC 和五分组评价	6
六、IC 加权 Top 10 组合回测	7
七、PyTorch LSTM 分钟涨跌预测	8
八、报告、验证和单元测试	10
九、复现方式和 README 中需注意的一处缺口	10
十、结论、边界与后续改进	11
附录 A：函数级代码索引	11
附录 B：最终产物清单	12

报告说明

本报告以用户提供的南京大学 Box 共享目录、本地 `readme (1).pdf`、当前工程代码和已生成结果为依据。共享目录中的文件为 `adjfactor.pkl`、`readme.pdf`、`readme (1).pdf`、`task.tar.gz` 和 `补丁.txt`；本地对应小文件的文件大小与网页清单一致。4.68 GB 的原始压缩包没有重复下载，因为数据已经在本地工程中完成处理。为避免泄露共享权限，报告不展开共享链接令牌。

本次在 2026-07-28 重新运行了 README 中已定义的单元测试和全量产物审计：4 项单元测试全部通过，`reports/validation.json` 状态为 `PASS`。因此，下文的数据规模和结果数字都来自当前已验证的产物，不是只根据文件名推测。

一、题目要求与完成状态

题目要求	实现位置	当前产物	状态
逐笔数据降采样为日频和分钟频，输出 11 个基础字段	<code>pipeline_code/downsample_tick_data_and_apply_adjustment.py</code>	日频 11 张 302 x 300 宽表；分钟频 3,322 张 253 x 300 宽表	完成
使用复权因子修正价格	降采样主入口；以及已有表的替代复权路径	<code>processed/daily/</code> 与 <code>processed/minute/</code>	完成
构建题目示例因子和 1 至 3 个自建因子	<code>pipeline_code/construct_factors.py</code>	1 个示例因子 + 3 个自建因子	完成
计算 IC、IR、ICIR、Rank IC、Rank IR、Rank ICIR 并做分层	<code>pipeline_code/evaluate_factors.py</code>	<code>evaluation/*.csv</code>	完成
1,000 万初始资金、Top 10、卖出万五，比较收盘和次日开盘调仓	<code>scripts/analyze_factors_backtest.py</code>	4 种“权重 x 交易口径”回测	完成
指出题目字面权重方案的问题并修正	<code>compute_weights()</code>	<code>leaky_same_day_ic</code> 对照 + <code>historical_20d_ic</code> 可交易修正	完成
用分钟线 LSTM 预测下一分钟涨跌	<code>scripts/train_lstm_pytorch.py</code>	<code>model.pt</code> 、预测、训练历史和指标	完成
对全部产物做结构和口径校验	<code>scripts/validate_outputs.py</code> 与 <code>tests/test_pipeline.py</code>	<code>validation.json = PASS</code> ，4 项测试通过	完成



整体逻辑是一条可追溯的管线：原始逐笔数据先经降采样和复权得到日频/分钟频面板，日频面板用来构建因子、评价和回测，分钟频面板用来训练 LSTM，最后由报告脚本和验证脚本收口。

二、项目结构和代码分工

```
data/TRADE/YYYYMMDD/ <code>.csv      # 
adjfactor.pkl                        # 
pipeline_code/                       # 
scripts/                             # K 
processed/daily/                     # 
processed/minute/ <field>/           # 
factors/ evaluation/ backtest/ lstm/ # 
reports/                             #
```

`pipeline_code/` 是 README 指定的核心路径。`downsample_tick_data_and_apply_adjustment.py` 与 `scripts/import_trade_data.py` 是同一降采样逻辑的描述性命名版和原脚本版；`apply_adjustment_factor_to_existing_tables.py` 与 `scripts/adjust_existing_processed.py` 也是同类对应。`scripts/analyze_factors_backtest.py` 保留了“构建因子 + 评价 + 回测”的合并入口；前两步已经拆分为 `construct_factors.py` 和 `evaluate_factors.py`，但回测函数仍在该合并脚本中。

三、数据降采样与复权

3.1 交易分钟网格：trading_minutes()

```
minutes = list(range(915, 926))
for hour, start, end in (
    (9, 30, 59), (10, 0, 59), (11, 0, 30),
    (13, 0, 59), (14, 0, 59), (15, 0, 0),
):
    minutes.extend(hour * 100 + minute for minute in range(start, end + 1))
```

这段代码按题目把交易日展开为 253 个分钟点：09:15-09:25 集合竞价、09:30-11:30 上午连续竞价、13:00-15:00 下午时段。午休时间不在索引中，15:00 作为收盘分钟被显式保留。单元测试同时检查了网格长度、时段边界和无重复。

3.2 复权因子：load_adjustment()

`adjfactor.pkl` 被读成“日期 x 股票”面板，日期统一为 YYYYMMDD，股票代码去掉交易所后缀。如果去后缀后出现重复代码，程序直接报错。因子表按日期排序，再向前、向后填充缺失值。在正式导入前，`import_data()` 还会一次性检查所有交易日和股票是否都被复权表覆盖。

价格与金额使用两种口径：

```
raw_price = df["Price"].astype(float) / 100.0
adjusted_price = raw_price * adjustment
amount = raw_price * df["Volume"]
```

- OHLC 使用 $\text{Price} / 100 * \text{adjfactor}$ ，用来缓解分红配股等事件导致的价格不连续。
- 成交额使用实际成交价 $\text{Price} / 100 * \text{Volume}$ ，不随复权价改变。

3.3 单只股票汇总：summarize_stock()

函数首先用明确数据类型读取 `Time`, `Price`, `Volume`, `BSFlag`，并严格检查列集合和 `BSFlag` 取值。稳定排序保证同一时间下的原始先后顺序不被改变。

输出字段	日频定义	分钟频定义
open	当日第一笔复权价	当分钟第一笔复权价
high / low	当日复权价最大/最小值	当分钟复权价最大/最小值
close	当日最后一笔复权价	当分钟最后一笔复权价
volume	全日成交量之和	分钟成交量之和
trade_count	全日逐笔记录数	分钟逐笔记录数
amount	实际价 x 成交量的全日合计	实际价 x 成交量的分钟合计
buy_volume/amount	只累加 BSFlag=0	只累加 BSFlag=0
sell_volume/amount	只累加 BSFlag=1	只累加 BSFlag=1

BSFlag=2 的集合竞价成交会进入总成交量和总成交额，但不进入主买/主卖子项。因此 `buy_volume + sell_volume <= volume` 是正常关系，不要强行令两者相等。

3.4 空值填充：不让未来价格倒灌

```
close_filled = (
    minute_by_field["close"].ffill()
    .fillna(previous_close)
    .bfill()
)
for field in PRICE_FIELDS:
    minute_by_field[field] = minute_by_field[field].fillna(close_filled)
for field in ZERO_FIELDS:
    minute_by_field[field] = minute_by_field[field].fillna(0)
```

日频无成交价格用前一日复权收盘价填充，量、笔数和金额填 0。分钟频先在当日向前填充，开盘前置空白再用昨日收盘填充；只有数据集第一天没有昨日价时才使用当日首个可观察价向后补齐。这个顺序避免常见错误：如果一开始就对全日 `bfill()`，会把当日稍后才出现的价格引入更早分钟。

3.5 全量导入：import_data()

`import_data()` 按日期循环，每天先建立“股票 x 11 字段”日频空表和 11 张“253 分钟 x 股票”空表，再逐股调用 `summarize_stock()` 回填。当日完成后立即写出 11 张分钟 CSV，所有日期完成后再把日频行堆叠成 11 张宽表。

已验证输出：

- 302 个交易日，300 只股票。
- 日频每字段为 302 x 300，共 11 张表。
- 分钟频每张为 253 x 300，302 日 x 11 字段 = 3,322 张表。
- 全量审计检查了 1,759,661 个无成交单元格的填充口径。

3.6 替代复权路径：apply_adjustment_factor_to_existing_tables.py

这个脚本只用于“已有同口径未复权 `processed` 表”的情况，与原始逐笔导入二选一，不应在主导入脚本之后重复执行。

- `load_wide()` 保留日期和分钟索引，并去掉 CSV 读入时可能出现的 .0。
- `adjustment_for()` 把复权表对齐到当前日期和股票，覆盖不全就报错。

- `rebuild()` 直接复制 7 个非价格字段；只对 OHLC 乘复权因子。
- 分钟价格借助成交量判断该分钟是否真有成交，再按前收口径重建空白。

3.7 合并日 K 线：`build_kline_csv.py`

`read_panel()` 将每个字段的日频宽表读入，强制股票代码补齐为 6 位字符串；`build_daily_kline()` 创建 `date x code` 多级索引，再将 11 个面板按相同顺序展平为 90,600 行长表。`validate_kline()` 检查代码长度、日期格式、唯一键、有限数、OHLC 大小关系、非负成交字段以及主买卖不超过总量/总额。

```
index = pd.MultiIndex.from_product([dates, codes], names=["date", "code"])
kline = index.to_frame(index=False)
for field in FIELDS:
    kline[field] = panels[field].to_numpy().reshape(-1)
```

输出 CSV 对非数字字段加引号，但下游仍应使用 `dtype={"code": str}` 读取，否则某些工具会把 000012 自动解释为整数 12。

四、因子构建

4.1 输入和输出

`construct_factors.py` 只读取日频 `close, high, low, amount, volume, buy_volume, sell_volume`。`load_wide()` 统一日期类型、排序并保留 6 位代码；`save_wide()` 把索引写回 YYYYMMDD。`build_factors()` 生成四张宽表，再通过 `stack()` 合并成 `factor_long.csv`，同时写出因子中文名、公式和类型元数据。

因子	公式和直觉	方向解读
<code>amount_mean_sd_log</code>	$\log(1 + \text{sd}(\text{MA1}, \text{MA5}, \text{MA10}, \text{MA20}))$ ；衡量成交额不同窗口均值的离散程度	数值越大，短中期成交额状态越不一致
<code>buy_sell_imbalance_5d</code>	5 日平均 $(\text{buy_volume} - \text{sell_volume}) / \max(\text{volume}, 1)$	正值表示近 5 日主动买入占优
<code>intraday_range_10d</code>	10 日平均 $(\text{high} - \text{low}) / \text{close}$	数值越大，日内波动越强
<code>reversal_5d</code>	$-(\text{close} / \text{close.shift}(5) - 1)$	近 5 日跌得越多，反转因子越高

题目示例因子的关键实现如下：

```
amount_means = [
    amount.rolling(window, min_periods=window).mean()
    for window in (1, 5, 10, 20)
]
stacked = np.stack([table.to_numpy() for table in amount_means], axis=0)
valid = np.isfinite(stacked).all(axis=0)
factor[valid] = np.log1p(np.std(stacked[:, valid], axis=0, ddof=1))
```

`min_periods=window` 保证每个移动均值只在窗口完整后才有值。四个时窗必须全部有限才计算标准差，`ddof=1` 使用样本标准差，`log1p()` 压缩偏态并避免对 0 直接取对数。

五、因子清洗、IC 和五分组评价

5.1 目标收益与面板清洗

因子评价的目标是下一交易日收盘到收盘收益：

```
next_return = close.shift(-1) / close - 1
panel = clean_factor_panel(factors, next_return)
```

`clean_factor_panel()` 把每个因子的宽表和收益宽表展开为 `date-code` 长表，然后按日做横截面处理：

1. 仅保留因子和未来收益都是有限数的样本，每日至少 20 只股票。
2. 用当日因子的 1% 和 99% 分位数做缩尾，减少极端值影响。
3. 用当日均值和样本标准差做 z-score，使不同因子可以参与后续线性合成。

5.2 IC、Rank IC、IR 和 ICIR

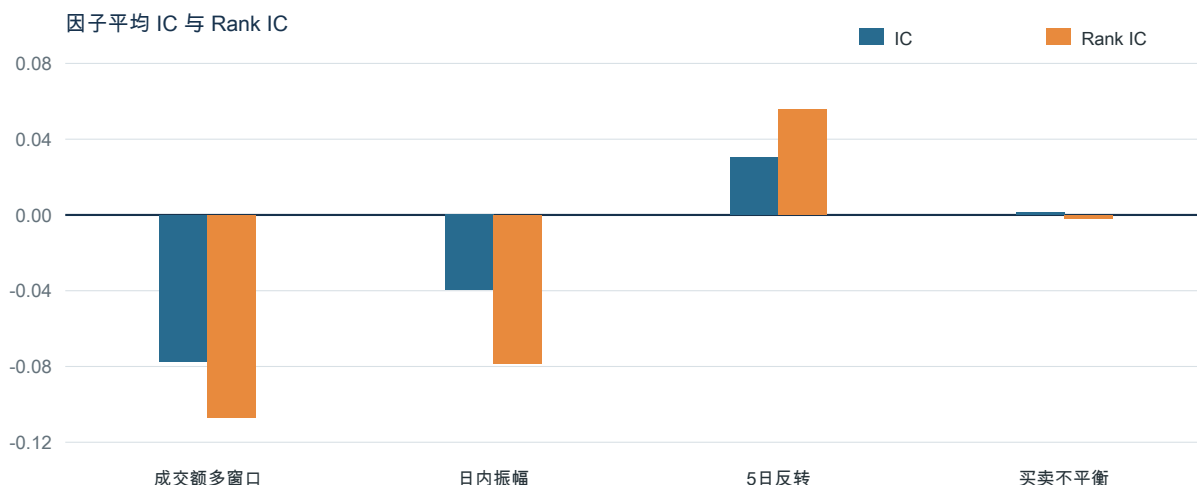
每个交易日、每个因子都计算两种横截面相关：Pearson 相关是 IC，Spearman 秩相关是 Rank IC。时序均值除以时序标准差得到 IR，再乘 `sqrt(252)` 得到年化 ICIR。

```
ic = np.corrcoef(x, y)[0, 1]
rank_ic = spearmanr(x, y).statistic
ir = ic_series.mean() / ic_series.std(ddof=1)
icir = ir * math.sqrt(252)
```

IC 的负号不等于因子无效，而是说因子值越大，下期收益倾向越低。如果这种负相关稳定，在组合中乘以负的历史 IC 后反而能正确翻转方向。

5.3 五分组和 Q5-Q1

对每日横截面的标准化因子做百分位排名，再用 `ceil(rank * 5)` 分到 Q1-Q5。每组等权收益通过连乘得到净值，Q5 - Q1 作为当日多空收益。这一步既看相关系数，也检查因子收益是否具有分层和单调性。



因子	IC	ICIR	Rank IC	Rank ICIR	Q5-Q1 累计收益
成交额多窗口均值离散度	-0.0774	-8.724	-0.1070	-11.711	-75.86%
10 日日内振幅	-0.0396	-4.180	-0.0783	-6.871	-41.95%
5 日反转	0.0303	3.199	0.0558	6.357	50.53%
5 日主动买卖不平衡	0.0016	0.265	-0.0017	-0.266	3.03%

结果中绝对 Rank IC 最大的是成交额多窗口因子，但方向为负。5 日反转因子为稳定正向，Q5-Q1 累计收益 50.53%。主动买卖不平衡因子的均值相关接近 0，单独使用时解释力较弱。

六、IC 加权 Top 10 组合回测

6.1 两种持有一期收益

```
if mode == "close_to_close":
    result = close.shift(-1) / close - 1
elif mode == "next_open_to_open":
    result = open_price.shift(-2) / open_price.shift(-1) - 1
```

- `close_to_close`：信号日 t 的收盘到 $t+1$ 收盘。程序将交易日记为 t ，等价于假设能用 t 日收盘数据并在同一收盘价成交，所以仍偏乐观。
- `next_open_to_open`：信号日 t 之后，在 $t+1$ 开盘买入，到 $t+2$ 开盘计算持有收益，交易日记为 $t+1$ 。

6.2 两种因子权重：错误对照与修正方案

```
if strategy == "leaky_same_day_ic":
    weights["weight"] = weights["raw_ic"]
elif strategy == "historical_20d_ic":
    weights["weight"] = weights.groupby("factor")["raw_ic"].transform(
        lambda s: s.shift(1).rolling(20, min_periods=20).mean()
    )
```

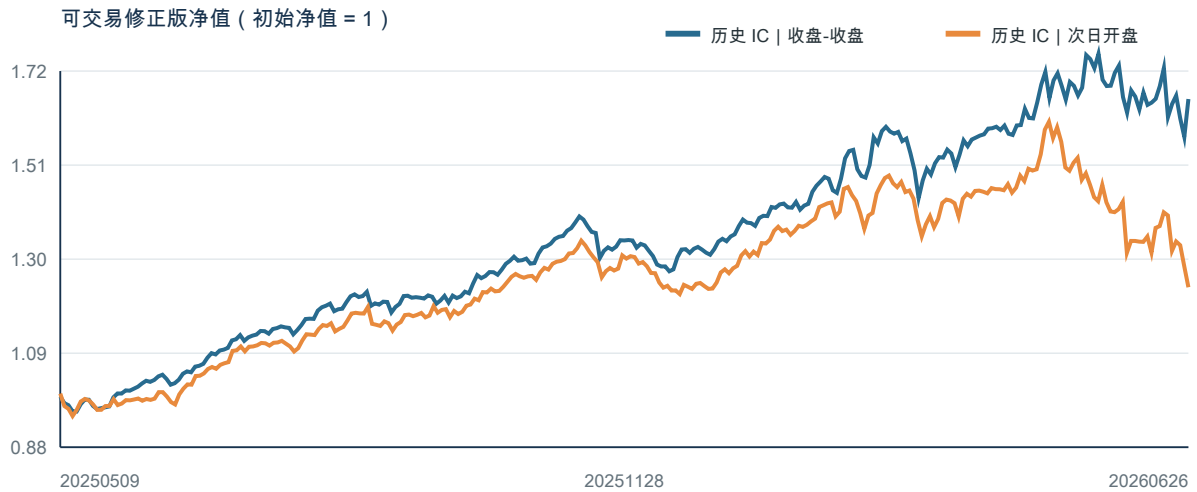
`leaky_same_day_ic` 用当日因子与未来持有一期收益计算当日 IC，再立即当作权重。因为调仓时并不知道未来收益，这是典型的前视偏差。该结果只能用来说明数据泄漏会把业绩夸大到何种程度。

`historical_20d_ic` 先对单因子 IC 整体滞后一天，再使用过去 20 日均值。所以任意信号日的权重都只由当时已知的历史数据决定，这才是可用于策略结论的版本。

6.3 信号、选股、换手和费用

每只股票的综合信号为“标准化因子 $\times$ 历史 IC 权重”在四个因子上求和。每日按信号从高到低选 10 只，目标权重各 10%。卖出换手由“当前漂移权重超出新目标权重的部分”累加得到，费用为期初净值 $\times$ 卖出换手 $\times$ 0.0005。买入费用按题目设定为 0。

```
signal = sum(factor_z * weight)
target = top_10_by_signal
fee = nav * sell_turnover * 0.0005
nav_end = (nav - fee) * (1 + target_return.mean())
```

权重方案	调仓口径	总收益	年化收益	年化波动	夏普	最大回撤
historical_20d_ic	close-to-close	65.75%	58.36%	21.06%	2.291	-10.55%
historical_20d_ic	next-open-to-open	23.72%	21.45%	21.89%	0.998	-23.00%
leaky_same_day_ic	close-to-close	2,206.28%	1,333.54%	23.31%	11.598	-4.35%
leaky_same_day_ic	next-open-to-open	2,482.51%	1,492.74%	24.55%	11.458	-3.14%

可信的主结论是 `historical_20d_ic`：收盘到收盘口径总收益 65.75%、夏普 2.291、最大回撤 -10.55%；次日开盘到开盘口径总收益 23.72%、夏普 0.998、最大回撤 -23.00%。两者差异说明成交时点假设对结果影响很大。同日 IC 方案的 2,000% 以上收益是未来信息泄漏的警报，不是一个超高收益策略。

七、PyTorch LSTM 分钟涨跌预测

7.1 样本和特征：`make_dataset()`

模型默认选取分钟表第一只股票，当前实验为 `000012`，使用前 60 个交易日。每一天单独生成长度 20 的滑动窗口，不让序列跨夜连接。标签是窗口后一分钟收盘价是否严格大于当前收盘价。

特征	计算	信息含义
<code>log_return</code>	<code>diff(log(close))</code>	最近一分钟收盘收益
<code>range</code>	<code>(high - low) / close</code>	分钟内波动幅度
<code>log_volume</code>	<code>log1p(volume)</code>	压缩后的成交活跃度
<code>buy_sell_imbalance</code>	<code>(buy - sell) / max(volume, 1)</code>	主动买卖方向
<code>time_position</code>	从 0 到 1 的日内位置	开盘/收盘时段效应

7.2 模型结构：`TorchLSTM`

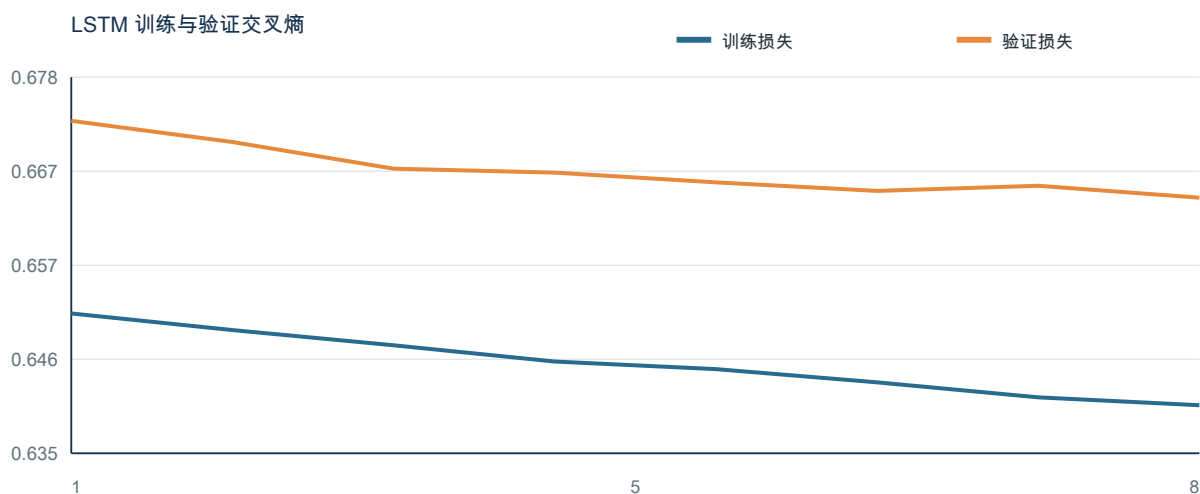
```
self.lstm = nn.LSTM(  
    input_size=5, hidden_size=16, batch_first=True  
)  
self.classifier = nn.Linear(16, 1)  
  
sequence, _state = self.lstm(x)  
logit = self.classifier(sequence[:, -1]).squeeze(-1)
```

输入形状为 `batch x 20 x 5`。单层 LSTM 输出整段隐状态，代码只取最后一个时刻，再用线性层输出一个 logit。训练时用 `BCEWithLogitsLoss`，预测时再通过 `sigmoid` 转为上涨概率。

7.3 时间切分、标准化和训练：train()

样本按原时间顺序切成 70% 训练、15% 验证和 15% 测试，没有随机打乱切分。特征均值和标准差只由训练集估计，然后应用到验证和测试集，避免预处理阶段泄漏未来信息。

优化器为 Adam，学习率 0.003，batch size 128，训练 8 轮。每个 batch 执行清梯度、前向传播、反向传播、梯度范数裁剪到 5.0、参数更新。每轮都计算训练集和验证集指标，保留验证损失最小的检查点。



分割	样本数	上涨占比	多数类基线	准确率	平衡准确率	AUC
训练	9,786	36.87%	63.13%	63.72%	52.07%	0.6028
验证	2,097	40.01%	59.99%	60.52%	51.95%	0.5772
测试	2,097	40.44%	59.56%	58.99%	51.05%	0.5654

测试集准确率 58.99% 低于“永远预测不涨”的 59.56% 多数类基线；平衡准确率 51.05% 只略高于随机水平，AUC 0.5654 说明有弱排序信号，但远不足以声称可交易。这一拓展的主要价值是完成了从分钟表、特征、序列、训练、检查点到测试输出的端到端流程。

7.4 输出文件

- `model.pt`：模型权重、特征列表、标准化参数、股票代码、序列长度和最优轮次。
- `model_metadata.json`：不含大型张量的可读元数据。
- `training_history.csv`：每轮训练/验证损失和指标。
- `metrics.csv`：训练、验证、测试三个分割的统一评价。
- `test_predictions.csv`：测试集每个时点的真实标签、上涨概率和 0/1 预测。

八、报告、验证和单元测试

8.1 build_report.py

该脚本不重算策略，而是读取 `factor_summary.csv`、`backtest_summary.csv`、`metrics.csv` 和 `factor_metadata.csv`，然后组装数据与字段说明.md 和 项目总报告.md。这种做法确保报告数字来自结果表，但也意味着运行它之前必须已经生成因子、回测和 LSTM 指标。

8.2 validate_outputs.py

全量审计不只检查文件是否存在，还检查日频/分钟频形状和对齐、有限数、OHLC 关系、无成交填充、K 线代码与行数、因子/回测/LSTM 结果行数、PyTorch 检查点键集合和特征列表，以及每次调仓是否恰好持有 10 只股票。审计通过后才写出 `validation.json`。

8.3 tests/test_pipeline.py

测试	检查目标
test_trading_minute_grid	253 个唯一分钟，包含 09:25、11:30、13:00、15:00，不包含午休
test_tick_aggregation_and_adjustment	用 3 笔人工逐笔数据核对 OHLC、量、笔数、金额、主买卖与复权
test_daily_kline_csv_preserves_codes_and_field_order	检查长表列顺序和 000001 前导零不丢失
test_pytorch_forward_backward_are_finite	检查 LSTM 前向、损失、梯度和一次参数更新都有限且有效

九、复现方式和 README 中需注意的一处缺口

项目应使用工程虚拟环境 `.venv/bin/python`，避免系统 Python 缺包或版本不一致。从原始数据完整复现的精简顺序为：

```
.venv/bin/python pipeline_code/downsample_tick_data_and_apply_adjustment.py \
--input data --adjfactor adjfactor.pkl --output processed
.venv/bin/python scripts/build_kline_csv.py
.venv/bin/python scripts/analyze_factors_backtest.py
.venv/bin/python scripts/train_lstm_pytorch.py \
--max-dates 60 --epochs 8 --device auto
.venv/bin/python scripts/build_report.py
.venv/bin/python scripts/validate_outputs.py
.venv/bin/python -m unittest discover -s tests -v
```

当前 README 的命令列表分别调用了 `construct_factors.py` 和 `evaluate_factors.py`，但没有显式调用回测生成步骤；而 `build_report.py` 又需要已存在的 `backtest/backtest_summary.csv`。因此，对于一个干净目录，需要补上 `scripts/analyze_factors_backtest.py`。该脚本会再次构建因子和评价结果，因此上面的精简路线不再单独执行两个拆分脚本。如果为了教学需要展示拆分步骤，可先运行构建与评价脚本，再运行合并脚本，但会重复前两步。

十、结论、边界与后续改进

这份作业的主任务和两个拓展都已经完成，而且产物结构、维度、填充逻辑和模型文件均通过自动化检查。因子端有明确结果：成交额多窗口因子呈稳定负相关，5 日反转因子呈正相关。组合端的可信版本在两种交易口径下都取得正收益，但次日开盘口径明显弱于收盘口径，说明结果对成交假设敏感。LSTM 完成了流程验证，但测试准确率低于多数类基线，不应包装成强预测能力。

当前结果还有四个主要边界：

- 1. 收盘信号按同一收盘价成交偏乐观，应进一步滞后信号或模拟收盘集合竞价。
- 2. 回测只计题目指定的卖出手续费，未计买入成本、滑点和冲击成本。
- 3. 数据不含行业、市值、停牌和涨跌停状态，因子未做行业/规模中性化和可交易过滤。
- 4. LSTM 只使用单只股票和前 60 日，下一步应扩展到多股票 walk-forward，并把概率阈值、换手、延迟和成本统一放入样本外评价。

附录 A：函数级代码索引

文件/函数	行号	作用
downsample...py / trading_minutes	25	构建 253 个交易分钟
downsample...py / load_adjustment	34	标准化复权因子日期和股票代码
downsample...py / summarize_stock	43	对单日单股逐笔数据同时生成日频和分钟频汇总
downsample...py / write_table	98	建目录并写宽表 CSV
downsample...py / import_data	103	驱动日期和股票循环，完成填充和全量输出
apply_adjustment...py / load_wide	21	读取已有宽表并整理索引
apply_adjustment...py / adjustment_for	32	生成与当前面板严格对齐的复权矩阵
apply_adjustment...py / rebuild	41	复制非价格字段，重建复权 OHLC 和空白填充
construct_factors.py / load_wide	41	读取日频字段面板
construct_factors.py / save_wide	50	统一写出日频因子宽表
construct_factors.py / build_factors	58	计算四个因子并写出宽表、长表和元数据
evaluate_factors.py / load_factor_panels	23	按元数据顺序读取因子
evaluate_factors.py / safe_ratio	35	安全计算均值/标准差及年化比率
evaluate_factors.py / max_drawdown	42	用历史峰值计算最大回撤
evaluate_factors.py / clean_factor_panel	49	合并未来收益，按日缩尾和标准化
evaluate_factors.py / evaluate	82	计算 IC、Rank IC、五分组、多空和汇总指标
analyze_factors_backtest.py / returns_for_mode	223	定义收盘或次日开盘持有期收益
analyze_factors_backtest.py / compute_weights	237	计算含泄漏对照或滞后 20 日历史 IC 权重

文件/函数	行号	作用
analyze_factors_backtest.py / simulate	260	因子合成、Top 10 选股、换手、费用和净值递推
analyze_factors_backtest.py / run_backtests	314	运行 2 x 2 回测并输出明细、持仓、权重和汇总
analyze_factors_backtest.py / write_analysis_report	363	从结果表生成因子与回测 Markdown 报告
build_kline_csv.py / read_panel	31	读取宽表并保留代码前导零
build_kline_csv.py / validate_kline	44	检查长表标识符、数值和 OHLC/成交逻辑
build_kline_csv.py / build_daily_kline	82	把 11 张日频宽表合并为日 K 长表
TorchLSTM.__init__ / forward	26/33	定义 LSTM + 线性二分类头及前向传播
train_lstm_pytorch.py / seed_everything	38	固定 Python、NumPy 和 PyTorch 随机种子
train_lstm_pytorch.py / resolve_device	46	按 MPS、CUDA、CPU 选择或检查运行设备
train_lstm_pytorch.py / read_code_column	60	从单日分钟宽表只读一只股票
train_lstm_pytorch.py / make_dataset	65	构建 20 x 5 序列、下分钟标签和时点元数据
train_lstm_pytorch.py / predict_proba	104	分 batch 推理并返回上涨概率
train_lstm_pytorch.py / score	116	计算交叉熵、准确率、平衡准确率和 AUC
train_lstm_pytorch.py / train	128	切分、标准化、训练、选模、评价并保存全部产物
build_report.py / main	19	从 CSV 结果组装项目中文总报告
validate_outputs.py / validate	22	执行全量结构、数值、填充、持仓和模型审计

附录 B：最终产物清单

- 日频与分钟频数据：[processed/](#)
- 合并日 K 长表：[kline_data/daily_kline.csv](#)
- 因子宽表、长表和元数据：[factors/](#)
- IC、分组、多空和汇总：[evaluation/](#)
- 组合净值、持仓、IC 权重和汇总：[backtest/](#)
- PyTorch 模型、训练历史、测试预测和指标：[lstm/](#)
- 生成报告和审计摘要：[reports/](#)